

온프레미스 SLM 구축 실전 가이드: 10원칙

외부 API 없이 내부에서 모델·인프라·서빙·운영 전부 구축한 실전 경험

SLM은 만능이 아니다. 복잡한 업무 설계와 시스템적 방어, 수치 기반 운영이 핵심이다.

AI 시스템 구축 실전 사례 공유

목차

01 배경과 도전과제

- 외부 API 사용 불가 환경
- 내부 구축의 현실
- 현업 기대와 SLM 한계
- 10가지 실전 교훈 개요

02 교훈 1-2: 간단한 작업과 4단계 설계

- SLM에 간단한 일만 맡기기
- 작업 절차 4단계 설계
- IT 헬프데스크 챗봇 사례
- 라우팅과 쿼리 분리 전략

03 교훈 3-5: 출력 형식과 프롬프트 관리

- JSON vs 마크다운 출력
- 문체 구체화 방법
- 모델별 프롬프트 관리
- 파싱 실패 방어 전략

04 교훈 6-8: 입력 형태와 모델 분리 전략

- Chat Template 포맷 이슈
- 무한 토큰 방어 방법
- 역할별 모델 분리 운영
- 캐스케이드 구조 설계

05 교훈 9-10: 수치 기반 운영과 모니터링

- GPU·지연·동시성 측정
- 수치 기반 인스턴스 산정
- trace_id 기반 모니터링
- 데이터 기반 의사결정

06 결론 및 핵심 메시지

- 바쁘면 놓치는 것들
- SLM은 도구일 뿐
- 업무 설계 최적화 우선
- 모니터링의 필수성

온프레미스 SLM 구축 배경과 도전과제

외부 API를 사용할 수 없는 환경

- 보안 정책으로 외부 API 호출 차단
- 모든 AI 기능을 내부에서 구현 필요
- 클라우드 서비스 의존 불가
- 온프레미스 환경에서 완전한 자체 구축

모델·인프라·서빙·운영 전부 내부 구축

- 모델 선정과 파인튜닝 직접 수행
- GPU 인프라 설계 및 구축
- 서빙 엔진(vLLM 등) 설정 및 운영
- 전체 파이프라인 내부 관리 체계 확립

현업의 기대와 SLM의 현실

- 현업: '버튼 하나로 자비스처럼' 기대
- 실제: SLM은 만능이 아님
- 복잡한 작업 처리의 한계 명확
- 업무 설계와 시스템 방어가 핵심

실전에서 배운 10가지 교훈 개요

- 간단한 작업 할당과 4단계 설계
- 출력 형식과 프롬프트 최적화
- 입력 형태 검증과 모델 분리
- 수치 기반 운영과 모니터링 우선

교훈 1: SLM에는 간단한 일만 맡겨라

복잡한 작업을 한 번
에 시키면 안 되는 이
유

- 요약+판단+정리+근거를 동시 요청
- 포맷 흔들림과 규칙 망각 발생
 - 재시도 비용 누적
- 단일 작업 할당 필요

'Lost in the
Middle' 현상의 실체

- 프롬프트가 길어질수록 발생
- 중요한 규칙이 묻히는 현상
- 앞쪽 규칙은 따르나 뒤쪽 무시
 - 체감 수준이 매우 높음

장애 보고서 자동 생
성 실패 사례

- 로그 요약→원인 판단→양식 정리→근거 첨부
- 단일 프롬프트로 요청 시 실패
- 뒤쪽 규칙 절반 이상 무시
- 사람이 재작업 필요

코드와 룰 기반 처리
후 SLM 활용

- 로그 요약: 정규식+통계 코드 처리
- 원인 판단: 룰 기반 1차 분류
- SLM: 정리된 사실을 양식에 채우기만
 - 재시도율 대폭 감소

교훈 2: 작업 절차를 4단계로 설계하라

A단계: 라우팅

- 요청 분류 단계
- 규칙/키워드로 처리
- 모델은 카테고리 선택만
- 모델 호출 최소화

B단계: 쪼개기

- 복합 질문 분리
- 쿼리 2~4개로 나눔
- 각각 한 가지 목적만
- 명확한 작업 단위 확보

C단계: 도구 호출

- 확정 정보 확보 단계
- DB·검색·API 활용
- 사실을 먼저 만들
- 할루시네이션 방지

D단계: 생성

- SLM 정리 단계
- 고정된 마크다운 포맷
- 내용 채우기만 수행
- 포맷 깨짐 최소화

핵심 포인트

- 프롬프트 최적화가 아님
- 업무 설계 최적화가 핵심
 - 비용 절감 효과 큼
 - 안정적인 품질 확보

4단계 설계 실전 사례: IT 헬프데스크 챗봇

A단계: 키워드 매칭으로 카테고리 분류

- 사용자 질문: VPN 연결 안 됨+재설치+승인 절차
- 키워드 매칭으로 IT인프라>네트워크 분류
- 모델 호출 없이 처리 완료
- 빠르고 정확한 라우팅 실현

B단계: 복합 질문을 3개 쿼리로 분리

- ①VPN 연결 장애 해결
- ②VPN 재설치 방법
- ③승인 절차 안내
- 각 쿼리는 독립적이고 명확한 목적 보유

C단계: 사내 위키 API 검색과 정보 확보

- 각 쿼리별로 사내 위키 API 검색
- 관련 문서 3건 확보
- 승인 절차는 그룹웨어 API에서 양식 링크 획득
- 확정된 사실 정보만 수집

D단계: 마크다운 포맷으로 정리

- SLM에게 3건 검색 결과 전달
- 정해진 헤더에 맞춰 정리 지시
- 문제해결·설치가이드·승인절차 구조화
- 할루시네이션과 누락 문제 해결

교훈 3: 출력은 JSON보다 마크다운 + 파싱

JSON 출력의 문제점

- JSON 강제 시 따옴표·콤마·괄호 하나로 파서가 터짐
- 닫는 따옴표 위치가 밀리면서 JSON 전체가 깨지는 현상
- 구조적 오류로 인한 파싱 실패 발생
- 복잡한 문자열 처리에서 특히 취약

실패 사례와 빈도

- 10건 중 2~3건 비율로 JSON 파싱 실패 발생
- 근거 값의 따옴표 문제로 전체 구조 손상
- 재시도 비용 증가와 서비스 안정성 저하
- 실제 운영 환경에서 빈번한 오류 경험

마크다운 헤더 기반 출력

- 마크다운 헤더(## 결론, ## 근거, ## 조치사항) 사용
- 정규식으로 섹션 분리 및 파싱 처리
- 구조가 단순하고 오류에 강한 포맷
- 모델이 더 안정적으로 출력 생성 가능

정규식 파싱과 폴백 처리

- 필수 키 누락 시 폴백 메시지 자동 삽입
- '파싱 실패 — 원문 확인 필요' 등으로 처리
- 포맷 깨짐으로 인한 재시도 대폭 감소
- 서비스가 멈추지 않는 안정적 운영 구현

교훈 4: 문체는 분해해서 지정하라

'정중하게'의 모호함이 초래한 문제

'정중하게 써라'는 지시는 범위가 너무 넓어서 모델이 해석하기 어렵습니다. 과도한 공손 표현이나 불필요하게 긴 문장이 생성되는 원인이 됩니다. 명확한 문체 기준 없이는 일관성 없는 출력이 반복됩니다.

과도한 공손문과 불필요한 장문 사례

'당신의 소중한 문의에 진심으로 감사드립니다. 해당 건에 대해 면밀히 검토한 결과...'처럼 지나치게 긴 인사말과 불필요한 수식어가 포함됩니다. 본문 전달보다 형식적 표현에 토큰이 낭비되는 문제가 발생했습니다.

격식체·간접화법·지시체로 구체화

격식체(~입니다), 간접화법(~로 확인되었습니다), 지시체(~하시기 바랍니다)로 구체적으로 지정합니다. 'VPN 연결 장애 원인은 방화벽 정책 변경으로 확인되었습니다'처럼 명확하고 간결한 문장 구조를 유도합니다.

토큰 소모 감소와 응답 시간 단축

불필요한 수식어와 인사말이 제거되어 토큰 소모가 크게 줄어듭니다. 응답 생성 시간도 단축되고, 사용자에게 핵심 정보를 빠르게 전달할 수 있습니다. 명확한 문체 기준으로 일관성도 향상됩니다.

교훈 5: 모델 계열 변경 시 프롬프트 재구성

같은 계열 내 프롬프트 호환성

- Qwen 계열 내에서는 프롬프트가 안정적으로 작동
- 동일 계열 모델 간 학습 패턴과 토큰화 방식 유사
- 버전 업그레이드 시에도 큰 수정 없이 사용 가능
- 계열 내 호환성은 비교적 높은 편

Qwen→Gemma
교체 시 성능 급락

- Qwen에서 Gemma로 모델 교체 후 분류 정확도 급락
- 5개 카테고리 중 2개를 지속적으로 혼동하는 현상
- 동일 프롬프트로는 성능 회복 불가능
- 모델 아키텍처 차이로 인한 근본적 문제

Few-shot 예시 배치
재구성

- Few-shot 예시의 배치 순서를 Gemma에 맞게 조정
- 구분자 스타일과 포맷을 모델 특성에 맞춰 재구성
- 프롬프트 재설계 후 정확도 회복 확인
- 모델별 최적화된 예시 구조 필요

모델 라인별 프롬프트
별도 관리

- 모델 라인별로 프롬프트를 따로 관리해야 함
- 공통 프롬프트 1개 고집은 운영에서 손해
- 각 모델의 특성을 반영한 맞춤형 설계 필수
- 버전 관리 시스템으로 프롬프트 이력 추적

교훈 6: 입력 형태 문제 점검하기

Chat Template 포맷 불일치 문제

프롬프트를 아무리 수정해도 성능이 개선되지 않을 때, 원인이 Chat Template 포맷 불일치인 경우가 있습니다. 모델이 기대하는 입력 형태와 실제 전달되는 형태가 다르면 규칙을 제대로 따르지 못합니다.

system 메시지 2개 분리 시 오류

system 메시지를 2개로 나눠 보냈을 때, Chat Template 포맷상 두 번째 system 메시지가 user 메시지로 합쳐지는 현상이 발생했습니다. 겉으로는 정상 응답이 나오지만 모델이 시스템 규칙을 무시하는 결과가 나타났습니다.

토크나이저 입력 문자열 최종본 로그 확인

서빙 엔진(vLLM 등)에 실제로 들어가는 토크나이저 입력 문자열 최종본을 로그로 출력해 확인합니다. 이를 통해 Chat Template 적용 후 실제 입력 형태를 파악하고 문제를 진단할 수 있습니다.

양자화 모델에서 특히 빈번한 이슈

원본 모델에서 잘 작동하던 프롬프트가 양자화 후 결과가 나빠지면 Chat Template 포맷 불일치를 의심해야 합니다. 양자화 과정에서 템플릿 처리 방식이 달라질 수 있어 특히 주의가 필요합니다.

교훈 7: 무한 토큰 문제의 시스템적 해결

repetition_penalty의 한계

- repetition_penalty · temperature로 감소 가능
- 완전 제거는 불가하며 정확도 저하 가능
- 파라미터만으로 근본 해결 불가

max_tokens의 실제 동작

- max_tokens는 클라이언트 측에서만 출력을 끊음
- 서버 GPU는 계속 토큰을 생성하며 자원 소모
- 실제 GPU 점유 시간은 줄어들지 않을 수 있음
- 시스템 레벨 제한이 필요한 이유

vLLM max-model-len 설정

- vLLM 서빙 시 --max-model-len으로 상한 제한
- 입력+출력 합산 토큰 수의 절대 상한 설정
- 예: --max-model-len 4096으로 전체 제한
- GPU 메모리 활용률도 함께 조정 필요

입력+출력 합산 상한 계산

- 상한 4096에서 입력 평균 3000이면 출력은 약 1096 한계
- VLM은 이미지 토큰까지 합산되므로 더 주의
- 입력 토큰 최대치를 미리 산정하고 설정
- 출력 가용 토큰 수 확보 후 상한 결정

교훈 8: 모델을 역할별로 분리 운영

8B 모델: 빠른 분류와 단순 작업

분류, 키워드 추출, 단순 요약 등 빠른 처리가 필요한 작업에 적합합니다. 저비용으로 높은 동시성을 처리할 수 있지만, 복잡한 추론은 약합니다. 전체 트래픽의 약 70%를 8B 모델이 소화했습니다.

30~80B 모델: 중간 수준 판단

문서 정리와 중간 수준의 판단 작업에 사용합니다. 8B보다 안정적이지만 동시성 부담이 있어 신중한 배치가 필요합니다. 복잡하지 않으면서도 정확도가 요구되는 작업에 적합합니다.

235B 모델: 복잡한 추론과 검증

최종 품질 검증과 복잡한 추론이 필요한 고위험 판단에 투입합니다. 정확도는 높지만 상시 운영에는 비용이 과다하므로, 꼭 필요한 경우에만 선택적으로 사용합니다.

라우팅 + 캐스케이드 구조 실전 사례

사용자 요청을 8B가 먼저 분류하고, 단순 질문은 8B가 직접 응답합니다. 복잡한 질문은 80B로, 고위험 판단은 235B로 라우팅합니다. 큰 모델로 가는 트래픽을 줄이는 것이 핵심이며, GPU 부족 시 탄력적으로 조정합니다.

교훈 9: GPU·지연·동시성은 수치로 결정하라

평균 호출 횟수 측정

- 요청 1건당 모델 호출 수를 카운트하여 측정
- 파이프라인의 복잡도를 판단하는 기준
- 감이 아닌 데이터로 시스템 구성 결정
- 평균 호출 횟수로 전체 부하 예측 가능

평균 가동 시간 산정

- 호출 1회당 GPU 점유 시간을 측정
- 모델 인스턴스 수 산정의 핵심 지표
- 하루 총 GPU 점유 시간 계산 가능
- 피크 시간대 동시 요청 수 고려 필수

응답 속도 분포 관리

- p50, p95, p99 지연 시간 기록
- SLA 준수 여부를 확인하는 지표
- 응답 속도의 분포를 파악하여 최적화
- 사용자 경험 품질 관리의 기준

실제 운영 예시

- 하루 500건 요청, 건당 2.3회 호출 시
- 호출당 1.2초 GPU 점유 → 총 23분
- 피크 시간 동시 요청 수로 인스턴스 산정
- 계산 없이 배포하면 큐 적체 또는 GPU 낭비

교훈 10: 모니터링은 기능보다 먼저 구축하라

기능 우선 개발의 문제

- 기능 중심 개발 시 원인 파악 불가
- 사후 대응으로 모두가 불만족한 상태 발생
- 모니터링 없이는 최적화 방향 결정 어려움

trace_id 기반 구성

- 요청 유입 시 trace_id 발급해 추적
- 라우팅·도구·모델 호출 단계별 기록
- 각 단계 소요시간과 성공/실패 저장

각 단계별 측정 항목

- 라우팅: 분류 결과, 소요시간
- 도구 호출: API 응답시간, 실패 여부
- 모델 호출: 토큰 수, 응답시간, 큐 대기시간
- 최종: 전체 소요시간, 재시도 횟수

실제 사례

- 초기 모니터링 없어 원인 불명
- trace_id 로깅으로 지연 원인 발견
- 전체 지연의 60%가 내부 검색 API 타임아웃

데이터 기반 의사결정

- 모니터링 데이터로 병목 지점 정확히 파악
- 라우팅 강화, 모델 분리, API 최적화 판단
- 감이 아닌 수치로 개선 방향 결정
- 지속적인 성능 개선의 기반 확보

결론: 실전에서 배운 핵심 메시지

온프레미스 SLM 구축은 모델 배포를 넘어
업무 설계·모니터링·수치 기반 운영의 통합입니다.
SLM은 도구이며 업무 설계 최적화가 우선입니다.
이 10가지 교훈을 실천하면 안정적 운영이 가능합니다.
